# Web Temps Réel — TP noté 1 : cahier des charges

Le TP noté 1 dure 1h30, sur le créneau 5, et porte sur les Server-Sent Events (séance 4).

## Contexte et sujet

Vous développez le suivi de commandes en direct d'une boutique en ligne. Une page de suivi montre au client l'état de sa commande et se met à jour sans rechargement ; les commandes sont créées et modifiées par l'API REST.

La boutique vend quelques produits en stock limité. Une commande porte sur un produit et une quantité. Elle passe par les statuts en préparation, expédiée puis livrée, et peut être annulée tant qu'elle n'est pas expédiée. La livraison est simulée par le serveur.

Le travail est individuel et part d'un projet vide.

Le projet part de ces trois types. Une commande annulée est conservée, avec le statut `cancelled`.

```
type User = {
  id: number;
  name: string;
  username: string;
  email: string;
};

type Product = {
  id: number;
  name: string;
  stock: number;       // quantité disponible
};

type Status = "preparing" | "shipped" | "delivered" | "cancelled";

type Order = {
  id: number;
  userId: User["id"];         // client qui commande
  productId: Product["id"];   // produit commandé
  quantity: number;
  status: Status;
  createdAt: string;          // date au format ISO
};
```

# Fonctionnalités attendues

Les exigences F1 à F10 sont obligatoires et notées sur 15 points ; les bonus en rapportent jusqu'à 5.

## Règles métier

- **F1.** La boutique compte au moins 3 produits et 2 utilisateurs. Utilisateurs, produits et commandes sont stockés avec JsonDB et survivent à un redémarrage du serveur.
- **F2.** Une commande indique un client, un produit et une quantité. Elle est refusée si un champ manque ou est invalide, si le client ou le produit n'existe pas ou si le stock est insuffisant. Sinon, le stock du produit diminue d'autant.
- **F3.** Une commande avance d'un statut à la fois : en préparation, expédiée, livrée. Aucun retour en arrière n'est possible.
- **F4.** Une commande en préparation peut être annulée, et son stock est rendu au produit. Une commande expédiée ou livrée ne peut plus l'être.
- **F5.** 20 secondes après son expédition, une commande passe seule au statut livrée.

## API REST

- **F6.** Vous concevez l'API qui couvre F1 à F4 : routes, verbes et codes de retour sont à votre charge. Chaque refus renvoie un code d'erreur adapté, et le README documente chaque route.

## Temps réel

- **F7.** Le serveur diffuse en SSE chaque création, changement de statut ou annulation de commande, ainsi que chaque variation de stock. Les événements sont nommés, et ceux des commandes sont distincts de ceux du stock.
- **F8.** Un abonné qui quitte la page (changement de page, fermeture de l'onglet) est retiré des abonnés.

## Page de suivi

- **F9.** Une page de suivi, ouverte pour une commande donnée, affiche son statut et se met à jour en direct, y compris lors de la livraison automatique de F5.
- **F10.** Cette page ne reçoit que les événements de sa commande : le tri est fait par le serveur, pas par le navigateur.

### Bonus

- **B1 (2 points).** Après une coupure réseau, la page de suivi récupère les événements de sa commande qu'elle a manqués, et seulement ceux-là.
- **B2 (1 point).** Une fois la commande livrée ou annulée, le flux de la page de suivi se termine et le navigateur ne tente plus de se reconnecter.
- **B3 (1 point).** Le serveur impose aux navigateurs un délai de 5 secondes avant chaque tentative de reconnexion.
- **B4 (1 point).** La connexion reste ouverte derrière un proxy qui coupe les connexions inactives au bout de 30 secondes.

## Contraintes techniques

- **Serveur :** Node.js 18 ou plus, Express 5 et TypeScript.
- **Docker :** le projet se lance avec la seule commande `docker compose up --build -d`, sur une machine où seul Docker est installé, et répond sur `http://localhost:3000`. Un projet qui ne se lance pas ainsi obtient 0.
- **SSE codé à la main :** pas de bibliothèque SSE, pas de socket.io, ni polling ni long polling.
- **Client :** HTML et JavaScript simple, servis par le serveur Express, sans framework front.
- **Typage :** les trois types fournis, utilisés côté serveur, et aucun `any`.
- **Dépôt :** `node_modules` exclu, et un `README.md` qui explique le lancement et documente l'API.

## Déroulé et rendu

Le TP est individuel et dure 1h30. Le rendu est une archive `.zip` du projet, sans le dossier `node_modules`, déposée sur MyGES avant la fin du créneau ; un dépôt plus tardif est accepté, avec une pénalité de 2 points par minute de retard.

Commitez au fil de l'eau sur votre dépôt Git. Le `README.md` donne le lien de ce dépôt, et liste les exigences et bonus terminés, et ceux qui ne le sont pas. Sans lien vers le dépôt Git dans le README, la note est divisée par 2.

Avant de déposer, vérifiez que le projet se lance depuis l'archive : décompressez-la, puis lancez `docker compose up --build -d`.

Les supports du cours sont autorisés ; l'IA générative ne l'est pas.

# Barème

Le TP est noté sur 15 points, et les bonus en ajoutent jusqu'à 5 : la note maximale est 20.

**Un projet qui ne se lance pas avec `docker compose up --build -d` obtient 0, bonus compris.**

- **Projet et qualité (2 points) :** Docker en place, persistance fonctionnelle (F1), code typé et lisible, commits réguliers.
- **Règles métier et API REST (3 points) :** stock contrôlé et mis à jour, transitions de statut et annulation respectées, code d'erreur adapté à chaque refus, routes documentées (F2 à F4, F6).
- **Flux SSE (7 points) :** flux conforme, événements nommés pour les commandes et le stock, abonnés retirés à la déconnexion, livraison automatique poussée par le serveur (F5, F7, F8).
- **Page de suivi (3 points) :** affichage du statut, mise à jour en direct et tri fait par le serveur (F9, F10).

**Bonus (5 points) :** B1 (2 points), B2, B3 et B4 (1 point chacun).